Was du ausprobiert hast

Physics-enhanced CNN · Chronologischer Rückblick · Mai 2024 bis März 2026

Ziel war, die zeitliche Entwicklung dreidimensionaler Temperaturfelder mit CNNs vorherzusagen und durch die Wärmeleitungsgleichung im Loss zu verbessern. Der rote Faden: erst Modellvarianten, dann die Konsistenz von Simulation und Physik-Loss, zuletzt gezielte Vergleiche verschiedener Residuen.

1 · Mai–August 2024: Vom statischen zum dynamischen CNN

Du startest mit Simulation, Normalisierung und statischen Modellen für feste Vorhersagezeiten. Ab Juni/Juli folgen dynamische CNNs mit Zeit als Eingang; im August variierst du die Zeiteinspeisung, reduzierst den Datensatz und änderst die Randbehandlung. Auch automatische Zeitableitungen werden erprobt; ein Codekommentar nennt Probleme beim Zusammenspiel mit numerischen Raumableitungen.

Ergebnis: Die gespeicherten Auswertungen zeigen deutliche Lernwirkung, aber starke Unterschiede zwischen Varianten. Über jeweils zehn Testfälle steigt der mittlere absolute Fehler des frühen dynamischen Modells von 47,33 bei 1 s auf 133,53 bei 10 s. Die Variante „latenttimeforrealthistime“ erreicht 26,62 bzw. 13,82. Das spricht für eine vielversprechende Zeiteinspeisung; wegen unterschiedlicher Trainingsbedingungen ist es kein isolierter Architekturbeweis. Werte in den rückskalierten Temperatureinheiten der damaligen Auswertung. [1]

2 · August 2025–Februar 2026: Wiederaufnahme und Baselines

2025 enthält der Verlauf vor allem README-Updates, im Januar 2026 folgt der HPC-Code-Stand. Im Februar vergleichst du statisch/dynamisch, frühe/späte Zeiteinspeisung und Training mit bzw. ohne Physik. Autograd wird entfernt; Run-IDs, Checkpoints und CSV-Metriken machen die Versuche nachvollziehbarer.

Ergebnis: Datengetriebene Modelle lernen gut. Beispiel vom 9. Februar: dynamisches CNN ohne Physik mit Validierungs-MSE 7,80e-7 nach zehn Epochen. Der entsprechende Physik-Lauf erreicht einen Gesamt-Loss von 5,65e-5; dieser enthält einen zusätzlichen Term und ist deshalb kein direkter Fehlervergleich. Ein belastbarer allgemeiner Vorteil der Physik ist hier noch nicht gezeigt. [2]

3 · 13.–20. Februar: Skalierung, Genauigkeit und PINN

Du korrigierst Temperatur- und Quellterm-Normalisierung, probierst float32/float64 und ersetzt die grobe Zeitableitung durch Differenzen benachbarter Modellvorhersagen. Parallel entsteht ein separates PINN mit punktweiser Abtastung.

Ergebnis: Mehr numerische Genauigkeit allein löst das Plateau nicht: mehrere dynamische Physik-Läufe bleiben bei ungefähr 0,023 Gesamt-Loss, auch mit float64. Beim PINN verändert sich der Validierungs-Gesamt-Loss über 50 Epochen nur von 0,02550 auf 0,02530; der PDE-Anteil bleibt bei etwa 0,0248. Das deutet in diesen Versuchen auf ein Problem der Formulierung/Optimierung hin, nicht bloß auf Rundungsfehler. [3]

4 · 23. Februar–Anfang März: Physik an die Simulation angleichen

Du korrigierst Laplace-Skalierung und Randbehandlung, wertest das Residuum nur im Inneren aus und verschiebst Quellen von den Rändern weg. Hinzu kommen hochauflösende Daten/Zarr, verschiedene Lernraten, kleinere Trainingsmengen, Loss-Gewichte, längere Läufe und Physics-only-Versuche.

Ergebnis: Auch nach den Korrekturen bleibt ein Zielkonflikt: Auf dem reduzierten Datensatz erreicht der kombinierte Lauf nach 50 Epochen einen Validierungs-MSE von 1,33e-5, der reine Datenlauf 7,40e-6. Kleinere Lernraten und alternative Zeiteinspeisung liefern keinen eindeutigen Durchbruch. Sehr kleine Physics-only-Residuen belegen zunächst nur, dass das jeweilige Residuum minimiert wird. [4]

Die entscheidenden Vergleiche im März

5 · 5.–13. März: Korrekte Zeitbasis und explizite Wärmequelle

Zeitschritte und Geometrie kommen nun aus Simulationsmetadaten; am ersten Zeitschritt wird der Anfangszustand als Vergangenheit verwendet. Du trennst Anfangstemperatur, Quellmaske und Quellfeld. Anschließend vergleichst du **model_past** (zwei Modellzustände), **gt_past** (vorheriger Simulationszustand) und **gt_future** (nachfolgender Simulationszustand).

Ergebnis: Im Debug-Vergleich nach 20 Epochen erreicht gt_future einen Validierungs-MSE von 1,10e-5, MSE-only 1,97e-5, gt_past 4,06e-5 und model_past 3,02e-4. gt_future ist hier vielversprechend; ein kleines modellbasiertes Residuum bedeutet dagegen noch keine genaue Temperaturvorhersage. [5]

6 · 23.–27. März: Vergleich auf steel_train_medium

Auf dem größeren Stahl-Datensatz vergleichst du dieselben Residuen mit der Daten-Baseline. Die folgende Tabelle verwendet einheitlich Epoche 14 und den **Validierungs-MSE**, nicht den gemischten Gesamt-Loss. Alle vier Runs verwenden laut Config PITCNN_dynamic, trotz „timefirst“ in manchen Config-Dateinamen. [6]

Versuch	Physik-Gewicht	Val.-MSE, Epoche 14
Nur Daten	0	2,19e-6
gt_future + Daten	1	3,72e-6
gt_past + Daten	1	5,41e-6
model_past + Daten	1	2,88e-4

Aussage: Die Daten-Baseline ist hier am genauesten. Die beiden GT-gestützten Varianten liegen deutlich näher daran als model_past. Unterschiedliche Seeds und damit potenziell unterschiedliche zufällige Datenaufteilungen begrenzen die Aussage; es ist kein kontrollierter Mehrfachvergleich.

7 · 27.–30. März: Schwächere Physik, Warmstart und Soft-Physics

Mit Physik-Gewicht 0,01 verbessert sich model_past auf 1,69e-5 MSE, bleibt aber hinter der Baseline. gt_past erreicht 4,23e-6; gt_future schwankt und endet bei 2,23e-5, obwohl zwischenzeitlich 2,39e-6 erreicht wurden. Nach 20 Epochen reinem Datentraining (2,25e-6) verschlechtern zehn zusätzliche Physik-Epochen den MSE auf 5,72e-6 mit gt_future bzw. 3,65e-6 mit gt_past. Auch das ist wegen neuer Seeds kein sauber isolierter Vorher-nachher-Vergleich. [7]

Soft-Physics verwendet den Laplace-Term des Anfangszustands und lässt den Quellterm weg. Nach dem dokumentierten Kanal-Fix erreicht der Lauf 2,99e-4 MSE: kein Vorteil gegenüber der Daten-Baseline. Die abschließenden Physics-only-Läufe haben jeweils 13 protokollierte Epochen; ihre Residuen fallen auf 5,76e-4 (gt_future) bzw. 4,29e-4 (gt_past). Ihr protokollierter MSE von null ist ein deaktivierter Messwert. Außerdem nutzen beide Modi Simulationsnachbarn und sind somit nicht datenfrei. [7]

Gesamtbild: Du hast vor allem gezeigt, wie entscheidend eine konsistente Diskretisierung, Quellbeschreibung und Gewichtung sind. Reines Datentraining bleibt in den jüngsten Vergleichen die stärkste Genauigkeits-Baseline. Ein allgemeiner Genauigkeits- oder Generalisierungsvorteil des zusätzlichen Physik-Loss ist bislang nicht belegt.

Quellen und Einordnung

Git bis 4867499 (27.03.2026), ergänzt um lokale Run-CSVs und den Status-Nachtrag vom 30.03.2026. Historische und lokale Ergebnisse sind getrennte Evidenzquellen; Metriken verschiedener Daten-/Code-Stände sind nicht unmittelbar vergleichbar. Neuere Zahlen sind Validierungswerte, kein unabhängiger Testnachweis.

[1] 310ebe5: plots/*/deviations.txt, evaluate.py; 16ab9e6, 7f9d778, 7913bab. [2] Runs d056d1/ca1b44 vom 09.02.; Commits c42bbbd, 03c4622. [3] fcaa462, d977bec, cd47e19; Runs 88ed97, eb07d7; pinn_20260220-154004. [4] 2237a9c, 8e85799, cb9994a; Runs d1a417/1b1203. [5] 5408f5d, fe8e545, 79056df, 59472c7; Debug-Runs vom 13.03. [6] steel_train_medium: e6895c, e2cca7, 7a6070, 7e2ea6. [7] Dortige Runs vom 27./30.03.; TRAINING_STATUS_2026-03-27.md; training/loss.py. Run-Kürzel bezeichnen die letzten sechs Zeichen der Ordnernamen; Werte aus metrics.csv bzw. loss_components.csv.